Image Processing and Vision Project

Guilherme Carreira ist106965
Vasco Rodrigues ist106180
João Pinho ist102765
Rafael Cruz ist106720

Introduction

This project addresses the problem of tracking and rectifying a paper document observed by a moving camera. Given a reference image of the document and a sequence of input frames, the objective is to estimate the geometric transformation that maps each frame to the reference view, enabling the generation of a perspective-corrected representation of the document over time.
In the first part of the project, the problem is formulated in a purely two-dimensional setting using RGB images. A planar homography is estimated for each frame based on feature correspondences and robust outlier rejection, allowing the document to be aligned with the reference template despite camera motion and viewpoint changes.
In the second part, the approach is extended to RGB-D data by incorporating depth information. This enables the reconstruction of 3D point correspondences and the estimation of rigid transformations between frames, improving robustness and allowing non-planar elements to be identified through geometric constraints.
Part 3 of the project is the removal of out of plane objects using our RGB-D data calculated in Part 2.
The project illustrates the use of both 2D and 3D geometric vision techniques for reliable document tracking and alignment under realistic imaging conditions.

1 Part 1: Homography Estimation from RGB images

1.1 Algorithmic Details

In this part, SIFT was used for feature detection and we used the Euclidean distance between descriptors to identify the two nearest-neighbors for each feature. These matches were filtered using Lowe's Ratio Test with a threshold of 0.75.

For Homography estimation, we implemented RANSAC with a dynamic iteration count that ensures that there is a 99% chance of finding a valid match ($P = 0.99$) and an inlier error threshold of 3 pixels.

For each iteration, we randomly sample 4 point correspondences to compute a hypothesis H, using the Direct Linear Transform (DLT) algorithm. We then choose the homography with the most inliers.

Finally, to ensure numerical stability, we normalize the data by shifting the data to its centroid and scaling so that the average distance from the centroid is $\sqrt{2}$ before computing the final Homography using SVD on all inliers from the best iteration (iteration with the most inliers).

1.2 Experiment Setup and Evaluation

As described in the project statement we can run the experiment with the command
`python main1.py path_to_refimg path_images_dir path_feature_dir path_output_dir`
The output directory will hold the results, which are one .mat file, containing the homography matrix, and one .jpg file, with the calculated homography applied to the current frame, for each frame.
When visualizing the results, we used both the capture1 and tesla datasets from jupyter, which yielded, for the most part, great results as seen on the figures in 1.3

1.3 Example Results

Template image:

Figure 1: Template Image

Below are some of the results of this pipeline:

Figure 2: Original Frame 09

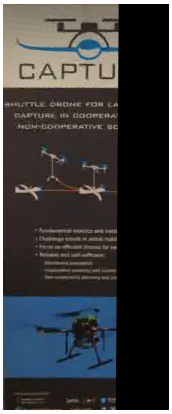

Figure 3: Rectified Frame 09

In Image 9, the template is correctly rectified despite the camera angle.

Figure 4: Original Frame 20

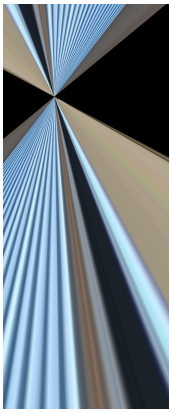

Figure 5: Rectified Frame 20

In Image 20, the template cannot be rectified as the number of outliers far exceeds the number of inliers, so the best homography is an incorrect one. This is a known RANSAC limitation.

2 Part 2: Incorporating Depth Information

2.1 Algorithmic Details

For each frame of the video, we run the following sequence:
Firstly, we perform SIFT feature detection and matching to the reference frame, with a k-nearest neighbours matcher with k = 2 to then perform Lowe's Ratio Test.
Then, valid matches are projected into 3D point clouds, and we perform RANSAC robust estimation to find the best rotation and translation.
In the RANSAC algorithm, we perform 1000 iterations, choosing three points per iteration and performing the Orthogonal Procrustes algorithm (after centering the data) on those points and their reference point cloud matches, selecting the group that has the most inliers, with an error threshold of 0.05.
Afterwards, we perform a final Orthogonal Procrustes Analysis with all inliers of the best group and their reference matches, and save the resulting rotation and translation.

2.2 Experiment Setup and Evaluation

As described in the project statement we can run the experiment with the command
`python main2.py path_to_refimgdir path_images_dir path_output_dir`
The output directory will hold the results, which are one .mat file for each frame, containing the rotation and translation arrays.
When visualizing the results, we used both the taag3d and plondres datasets from jupyter, which yielded well formed point clouds, indicating our algorithm was working as intended.

2.3 Example Results

This part's output is just a Rotation and a Transpose matrix, both saved in a .mat file.
In order to visualize the results, we drew the point clouds of all images of a sequence, rotated and translated to create a 3D point cloud of the whole scene.

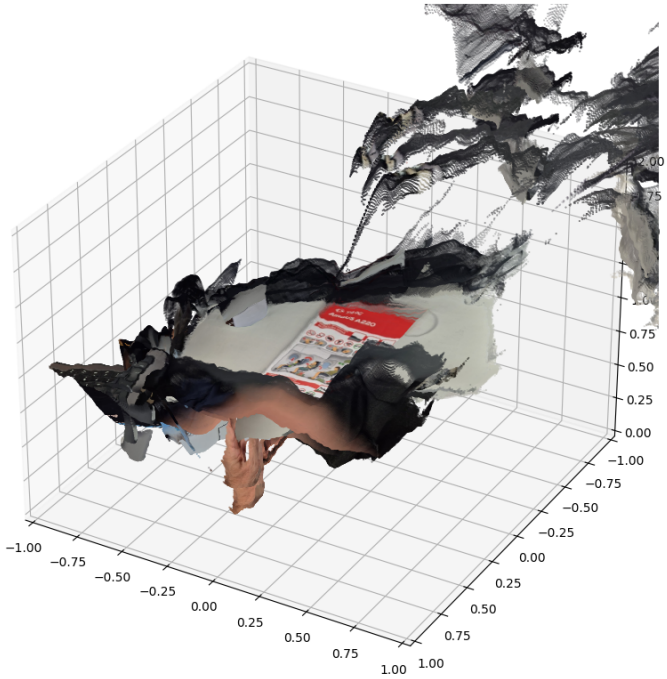

Figure 6: Aggregated point cloud of all sequence frames rotated and translated to be aligned

3 Part 3: Out of Plane Objects' Removal

For the last part of our project, we tried to implement RANSAC plane fitting on the results from part 2 in order to remove out of plane objects from our image.

3.1 Algorithmic Details

In this part, RANSAC was used with 1000 tries and an error threshold of 0.02.
For each iteration, we randomly sample 3 points and, if they're not colinear, calculate the plane formed by them, represented by the center point and normal vector.
We choose the plane with the most inliers.
Then, for each frame, we project the points into 3D, rotate and translate the frame's point cloud with the results from part 2, and remove points outside of the error threshold. Note that in order to achieve better results, we added the scale factor of each image as a result of part 2, along with the previously calculated rotation and translation.
The transformed 3D points from each frame are projected into the template image using the camera intrinsics, producing 2D pixel locations. To handle sparsity, each point is splatted to a small neighborhood, and a z-buffer is used to keep only the closest points at each pixel. The generated image is then saved.

3.2 Experiment Setup and Evaluation

We experimented with the taag3d dataset provided in jupyter.
Overall, the experiment was successful, but there is room for improvement. Further work could be a better scaling of the rotated image to be the same size as the template image, and perfecting the rotations and translations of Part 2, in order to create more consistent results.

3.3 Example Results

Template image:

Figure 7: Template image

Below are some of the results of this pipeline:
On the left the original frame, on the middle the result of the object removal over a black canvas and on the right the result over the template image.

Image 06 of the sequence is not occluded, and the results are satisfactory.

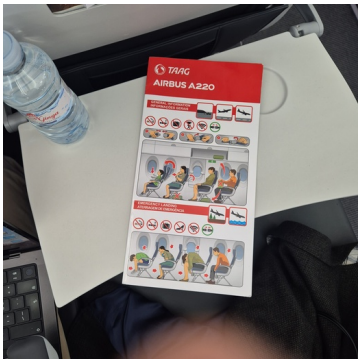

Figure 8: Original Frame 06

Figure 9: Results over Black canvas

Figure 10: Final frame 06

Image 09 has a hand blocking part of the template document, which is erased in the result. There is still some noise in the image.

Figure 11: Original Frame 09

Figure 12: Results over Black canvas

Figure 13: Final frame 09

In Image 11 the document is lifted from the plane's seat, but the results are still clear.

Figure 14: Original Frame 11

Figure 15: Results over Black canvas

Figure 16: Final frame 11